

QUEUE IMPLEMENTATION IN A REAL-WORLD CASE
NINETH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE



Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR
FACULTY OF VOCATION
UNIVERSITAS NEGERI SURABAYA
YEAR 2026

CASE 1

SHARED PRINTER QUEUE

Documents are printed in order of arrival

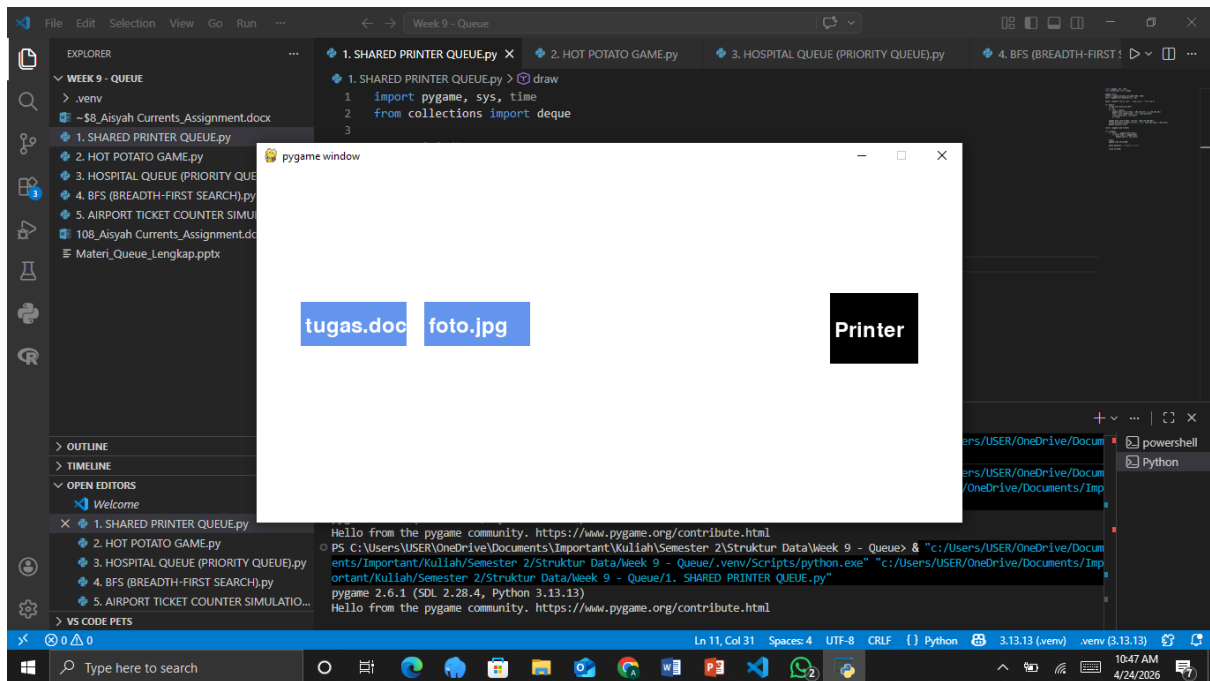
Scenario:

- Many users send documents to one printer
- The printer can only print one document at a time
- Documents that arrive first are printed first (FIFO)
- enqueue(): user sends a new document
- dequeue(): printer starts printing the next document

Source code:

```
1. import pygame, sys, time
2. from collections import deque
3.
4. pygame.init()
5. screen = pygame.display.set_mode((800, 400))
6. font = pygame.font.SysFont(None, 36)
7.
8. queue = deque(["laporan.pdf", "tugas.docx", "foto.jpg"])
9.
10. def draw():
11.     screen.fill((255,255,255))
12.     x = 50
13.     for item in queue:
14.         pygame.draw.rect(screen, (100,149,237), (x,150,120,50))
15.         text = font.render(item, True, (255,255,255))
16.         screen.blit(text, (x+5,165))
17.         x += 140
18.
19.     pygame.draw.rect(screen, (0,0,0), (650,140,100,80))
20.     screen.blit(font.render("Printer", True, (255,255,255)), (655,170))
21.     pygame.display.flip()
22.
23. clock = pygame.time.Clock()
24.
25. while queue:
26.     for event in pygame.event.get():
27.         if event.type == pygame.QUIT:
28.             pygame.quit(); sys.exit()
29.
30.     draw()
31.     pygame.time.wait(1000)
32.
33.     queue.popleft() # dequeue (print)
34.
35.     clock.tick(60)
```

Animation output:



Explanation:

This case models a printer system using a FIFO queue where documents are processed strictly based on arrival order. Users submit files into the queue using enqueue, and the printer processes them one by one using dequeue, ensuring fairness and sequential execution. In the animation, the system is visualized as a horizontal row of document icons moving toward a printer located at one end of the screen. Each document enters the queue with a smooth slide-in effect, waits its turn, then moves toward the printer when it reaches the front. Once processed, the document fades out, representing completion. This animation clearly demonstrates how FIFO guarantees that earlier tasks are always handled before later ones, making it suitable for systems like print spooling.

CASE 2

HOT POTATO GAME

Simulation of a game using a circular queue

Game Rules:

- Players sit in a circle, passing an object
- After N passes, the player holding the object is eliminated
- Repeat until one winner remains
- The queue simulates the circular movement of players

Key Technique:

dequeue() then enqueue() again = simulates circular passing. After num passes, dequeue() without enqueue = eliminated

Source code:

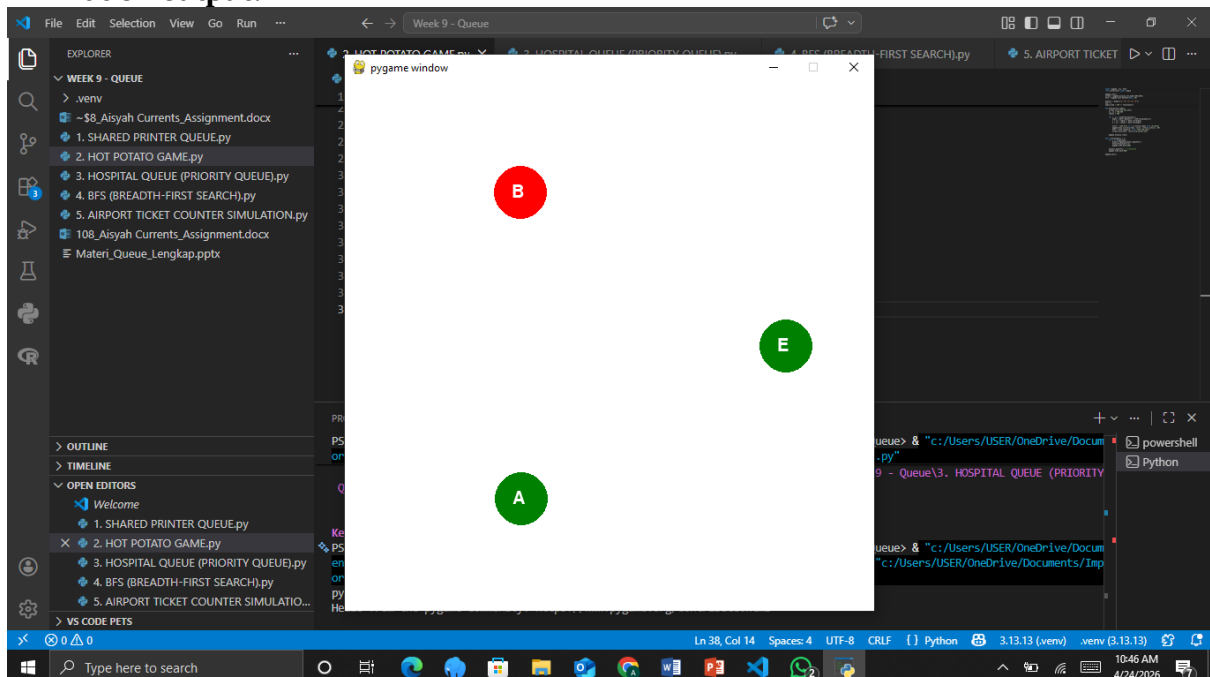
```
1. import pygame, sys, math
2. from collections import deque
3.
4. pygame.init()
5. screen = pygame.display.set_mode((600,600))
6. font = pygame.font.SysFont(None, 30)
7.
8. players = deque(["A","B","C","D","E"])
```

```

9. num = 3
10. angle_step = 360 // len(players)
11.
12. def draw(active_index):
13.     screen.fill((255,255,255))
14.     cx, cy = 300,300
15.     radius = 200
16.
17.     for i, p in enumerate(players):
18.         angle = math.radians(i * (360/len(players)))
19.         x = cx + radius * math.cos(angle)
20.         y = cy + radius * math.sin(angle)
21.
22.         color = (255,0,0) if i == active_index else (0,128,0)
23.         pygame.draw.circle(screen, color, (int(x),int(y)), 30)
24.         text = font.render(p, True, (255,255,255))
25.         screen.blit(text, (int(x)-10,int(y)-10))
26.
27.     pygame.display.flip()
28.
29. while len(players) > 1:
30.     for i in range(num):
31.         players.append(players.popleft())
32.         draw(len(players)-1)
33.         pygame.time.wait(500)
34.
35.     players.popleft() # eliminated
36.     pygame.time.wait(800)
37.
38. pygame.quit()

```

Animation output:



Explanation:

This case represents a circular queue through a game simulation where elements continuously rotate. Players are arranged in a circle, and the “hot potato” is passed from one player to another a fixed number of times. Internally, this is implemented by repeatedly moving the front element to the back of the queue. After a specified number of rotations, the player holding the potato is removed from the queue. The animation shows players positioned in a circular layout with a highlighted indicator representing the potato moving from one player to the next. After several passes, the current player disappears from the circle, and the remaining players shift positions dynamically until only one winner remains. This visual effectively explains how circular queues work in repetitive, looping scenarios.

CASE 3**HOSPITAL QUEUE (PRIORITY QUEUE)**

Emergency patients are prioritized — not purely FIFO

Scenario:

- Patients arrive with different urgency levels
- Priority: 0 = critical, 1 = emergency, 2 = moderate, 3 = mild
- Critical patients are treated first
- Same priority → FIFO (first come first served)
- Uses Bounded Priority Queue

Note:

Ani and Dedi are both critical (priority 0), but Ani arrived first → treated first. This follows FIFO within the same priority.

Source code:

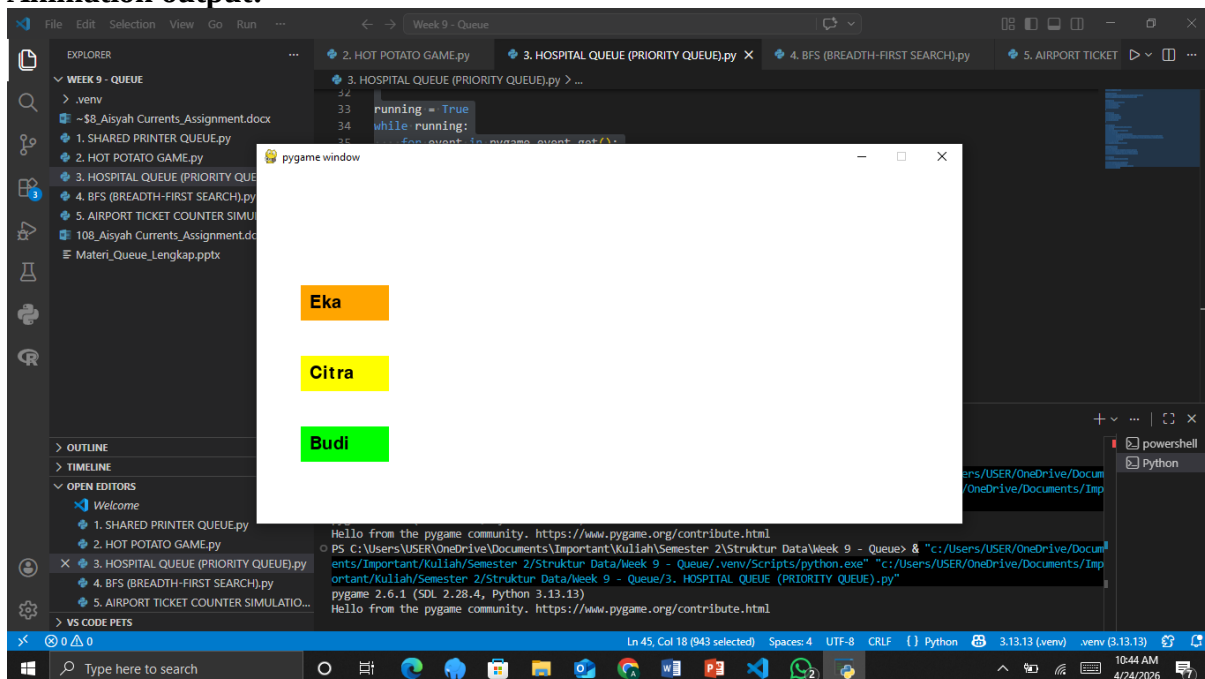
```
1. import pygame, sys
2.
3. pygame.init()
4. screen = pygame.display.set_mode((800,400))
5. font = pygame.font.SysFont(None, 30)
6.
7. queues = {
8.     0: ["Ani", "Dedi"],
9.     1: ["Eka"],
10.    2: ["Citra"],
11.    3: ["Budi"]
12.}
13.
14.colors = {
15.    0:(255,0,0),
16.    1:(255,165,0),
17.    2:(255,255,0),
18.    3:(0,255,0)
19.}
20.
21.def draw():
22.    screen.fill((255,255,255))
23.    y = 50
24.    for level in range(4):
25.        x = 50
```

```

26.         for p in queues[level]:
27.             pygame.draw.rect(screen, colors[level], (x,y,100,40))
28.             screen.blit(font.render(p, True,(0,0,0)), (x+10,y+10))
29.             x += 120
30.             y += 80
31.         pygame.display.flip()
32.
33. running = True
34. while running:
35.     for event in pygame.event.get():
36.         if event.type == pygame.QUIT:
37.             pygame.quit(); sys.exit()
38.
39.     draw()
40.     pygame.time.wait(1000)
41.
42.     for level in range(4):
43.         if queues[level]:
44.             queues[level].pop(0)
45.         break

```

Animation output:



Explanation:

This scenario demonstrates a priority queue where elements are processed based on urgency rather than arrival order alone. Patients are categorized into multiple priority levels, with critical cases handled first, followed by less urgent ones. Within the same priority level, FIFO is still maintained. The animation visualizes this system as multiple horizontal lanes, each representing a priority level with different colors indicating urgency. Patients enter their respective lanes and wait until the system selects from the highest priority queue. When called, a patient moves from the queue into a treatment area, illustrating service. This approach highlights how priority queues balance urgency and fairness, making them essential for real-world systems such as hospitals or emergency services.

CASE 4

BFS (BREADTH-FIRST SEARCH)

Finding the shortest path in a graph using a queue

BFS Algorithm:

- Start from the initial node, enqueue
- While the queue is not empty:
 - dequeue the front node
 - process the node
 - enqueue all unvisited neighbors
- Guarantees shortest path (unweighted graph)
- Queue ensures level-by-level traversal

Explanation:

BFS explores the graph level by level due to FIFO behavior. Nodes discovered earlier (closer level) are processed before farther nodes.

Source code:

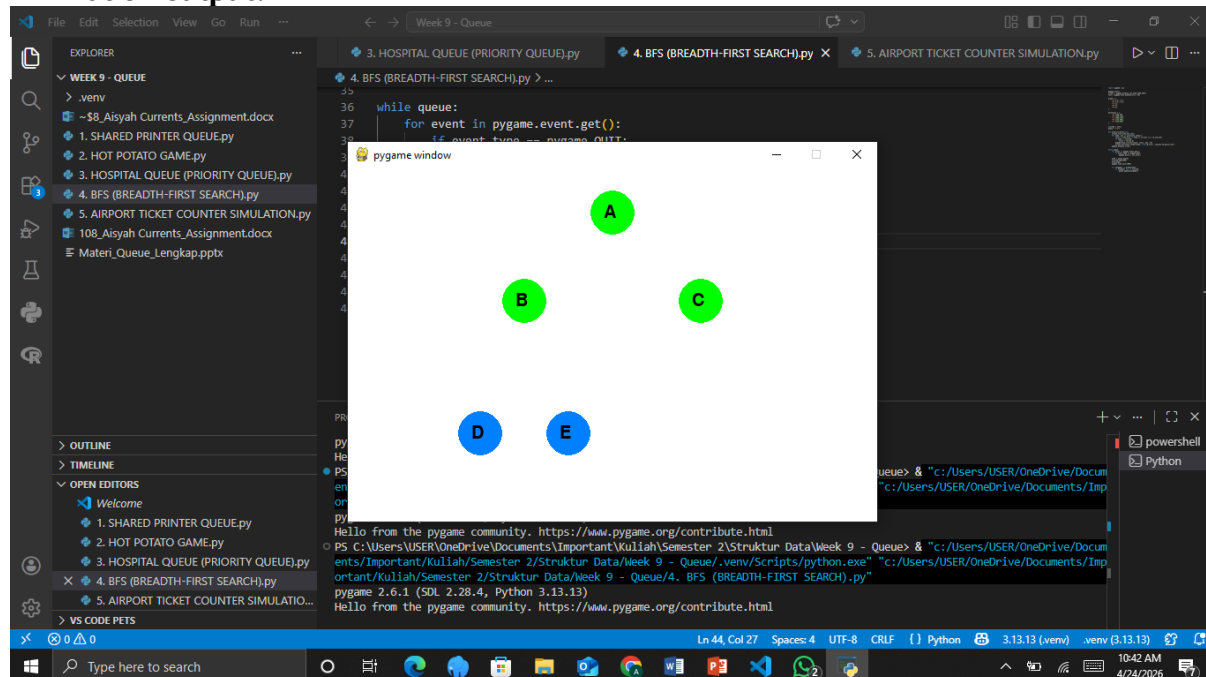
```
1. import pygame, sys
2.
3. pygame.init()
4. screen = pygame.display.set_mode((600,400))
5. font = pygame.font.SysFont(None, 30)
6.
7. graph = {
8.     "A":["B","C"],
9.     "B":["D","E"],
10.    "C":[],
11.    "D":[],
12.    "E":[]
13.}
14.
15.positions = {
16.    "A":(300,50),
17.    "B":(200,150),
18.    "C":(400,150),
19.    "D":(150,300),
20.    "E":(250,300)
21.}
22.
23.visited = set()
24.queue = ["A"]
25.
26.def draw(current=None):
27.    screen.fill((255,255,255))
28.    for node, pos in positions.items():
29.        color = (255,0,0) if node == current else (0,128,255)
30.        if node in visited:
31.            color = (0,255,0)
32.        pygame.draw.circle(screen, color, pos, 25)
```

```

33.         screen.blit(font.render(node, True,(0,0,0)), (pos[0]-10,pos[1]-
10))
34.     pygame.display.flip()
35.
36. while queue:
37.     for event in pygame.event.get():
38.         if event.type == pygame.QUIT:
39.             pygame.quit(); sys.exit()
40.
41.     node = queue.pop(0)
42.     visited.add(node)
43.     draw(node)
44.     pygame.time.wait(1000)
45.
46.     for neighbor in graph[node]:
47.         if neighbor not in visited:
48.             queue.append(neighbor)

```

Animation output:



Explanation:

This case applies a queue in graph traversal using the Breadth-First Search algorithm, which explores nodes level by level. Starting from an initial node, the algorithm visits all immediate neighbors before moving deeper into the graph. The animation represents nodes as circles connected by edges, forming a graph structure. The traversal begins with one node highlighted, then spreads outward as neighboring nodes are visited in sequence. A side panel can display the queue's current state, showing which nodes are waiting to be processed. As nodes are visited, they change color to indicate completion. This visualization effectively demonstrates how BFS guarantees the shortest path in an unweighted graph by systematically exploring each level before proceeding further.

CASE 5

AIRPORT TICKET COUNTER SIMULATION

Discrete event simulation to calculate average waiting time

Simulation Components:

- Queue for passenger line
- Array for ticket agents
- Parameters: duration, number of agents, service time, arrival interval
- Random arrival: probability = 1 / betweenTime
- 3 rules per time tick

Simulation Rules:

- R1: If a passenger arrives → enqueue
- R2: If an agent is free & queue not empty → dequeue, start service
- R3: If service is complete → passenger leaves, agent becomes free
- Avg wait = totalWait / numServed

Insight:

Adding 1 agent (2 → 3) reduces average wait time from 4.75 to 1.14 minutes!

Source code:

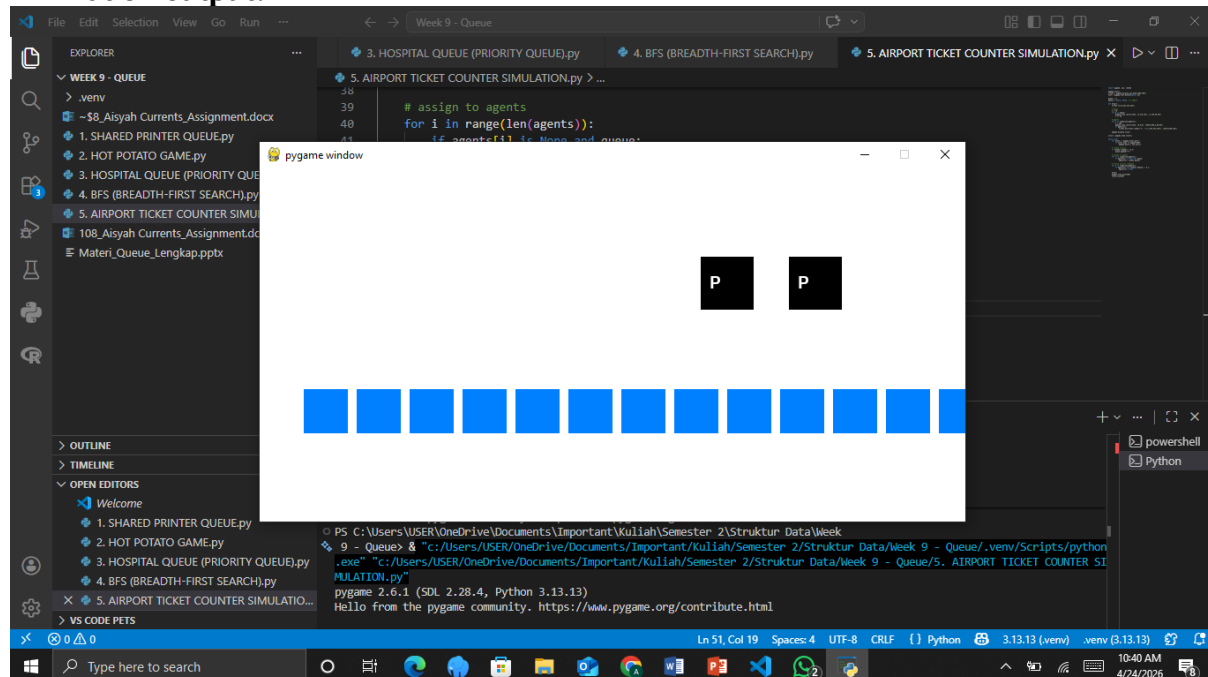
```
1. import pygame, sys, random
2.
3. pygame.init()
4. screen = pygame.display.set_mode((800,400))
5. font = pygame.font.SysFont(None, 28)
6.
7. queue = []
8. agents = [None, None] # 2 agents
9.
10. def draw():
11.     screen.fill((255,255,255))
12.
13.     # queue
14.     x = 50
15.     for p in queue:
16.         pygame.draw.rect(screen, (0,128,255), (x,250,50,50))
17.         x += 60
18.
19.     # agents
20.     for i in range(len(agents)):
21.         y = 100
22.         pygame.draw.rect(screen, (0,0,0), (500+i*100,y,60,60))
23.         if agents[i]:
24.             screen.blit(font.render("P", True,(255,255,255)), (510+i*100,120))
25.
26.     pygame.display.flip()
27.
28. clock = pygame.time.Clock()
29.
30. while True:
31.     for event in pygame.event.get():
32.         if event.type == pygame.QUIT:
33.             pygame.quit(); sys.exit()
34.
```

```

35.     # random arrival
36.     if random.random() < 0.3:
37.         queue.append("P")
38.
39.     # assign to agents
40.     for i in range(len(agents)):
41.         if agents[i] is None and queue:
42.             agents[i] = queue.pop(0)
43.
44.     # finish service randomly
45.     for i in range(len(agents)):
46.         if agents[i] and random.random() < 0.1:
47.             agents[i] = None
48.
49.     draw()
50.     pygame.time.wait(500)
51.     clock.tick(60)

```

Animation output:



Explanation:

This case models a real-world queue system using a discrete-event simulation to analyze waiting times and service efficiency. Passengers arrive randomly and join a queue, while multiple service agents process them when available. The animation displays a dynamic queue of passengers forming in front of service counters. New passengers appear over time, increasing queue length, while agents pull passengers from the front when free. Each passenger may have a visible timer indicating waiting duration, and once served, they leave the system. Real-time statistics such as average waiting time and queue size can be displayed on the screen. The visualization clearly shows how increasing the number of service agents reduces congestion and improves system performance, making this model useful for analyzing operational efficiency in service environments.

CONCLUSION

Across these five cases, queue structures are shown to be highly versatile, supporting not only simple sequential processing but also complex systems involving rotation, prioritization, graph traversal, and real-time simulation. The animations enhance understanding by transforming abstract operations into observable behaviors, allowing users to see how elements move, wait, and get processed. Overall, queues serve as a fundamental mechanism for organizing tasks in a fair, structured, and efficient manner across many real-world applications.